4. Keylime Agent 컴포넌트 상세 분석

4.1 Agent의 역할과 책임

Keylime Agent는 검증 대상 시스템에서 실행되는 핵심 컴포넌트로, TPM(Trusted Platform Module)과 직접 통신하여 시스템의 무결성 증명을 제공하는 역할을 담당합니다.

4.1.1 주요 책임

1. **TPM 통신 관리**: 하드웨어 또는 소프트웨어 TPM과의 직접적인 인터페이스 제공
2. **무결성 증명 생성**: PCR(Platform Configuration Register) 값과 TPM Quote 생성
3. **암호화된 페이로드 수신**: Tenant로부터 전송된 암호화된 데이터의 안전한 수신 및 복호화
4. **IMA 측정 로그 제공**: 파일 시스템 무결성 측정 데이터 수집 및 전송
5. **상호 TLS 인증**: mTLS를 통한 안전한 통신 채널 유지

4.1.2 시스템 내 위치

Agent는 Keylime 아키텍처에서 다음과 같은 위치를 차지합니다:

```
┌─────────────────┐  ┌─────────────────┐

|    Registrar    | |     Verifier    |

|    (등록 관리)   | |   (지속적 검증)  |

└─────────────────┘  └─────────────────┘

         |                    |

         | 등록/키 관리         | 주기적 검증
```

```
        |                    |
┌──────────────────────────────────┐
| Agent |
| (검증 대상 시스템) |
└──────────────────────────────────┘
|
┌──────────────────┐
| TPM |
| (하드웨어 보안) |
└──────────────────┘
```

4.2 TPM과의 통신 방식 분석

Agent는 `keylime/tpm/tpm_main.py` 의 `Tpm` 클래스를 통해 TPM과 통신합니다.

4.2.1 핵심 TPM 통신 메서드

4.2.1.1 AIK 암호화 (`encrypt_aik_with_ek`)

```
@staticmethod
def  encrypt_aik_with_ek(uuid: str, ek_tpm: bytes, aik_tpm: bytes) -> Optional[T
"""EK(Endorsement Key)를 사용하여 AIK(Attestation Identity Key) 암호화"""
# AIK 이름 생성
aik_name = tpm2_objects.get_tpm2b_public_name(aik_tpm)
# 32바이트 랜덤 챌린지 생성
```

```
challenge_str = tpm_util.random_password(32)

challenge = challenge_str.encode()

# MakeCredential을 사용한 자격 증명 생성

credentialblob = tpm_util.makecredential(ek_tpm, challenge, bytes.fromhex(aik_na

keyblob = base64.b64encode(credentialblob)

return (keyblob, key)
```

4.2.1.2 TPM Quote 생성 과정

Agent는 `do_quote()` 메서드를 통해 TPM Quote를 생성합니다:

1. **Nonce 생성**: 20바이트 랜덤 nonce 생성
2. **Quote 요청**: `/v{version}/quotes/identity?nonce={nonce}` 엔드포인트 호출
3. **응답 데이터**:

- `quote` : TPM에서 생성된 서명된 PCR 값
- `pubkey` : 공개키 정보
- `hash_alg` : 사용된 해시 알고리즘
- `enc_alg` : 사용된 암호화 알고리즘

4.2.2 TPM 통신 보안

- **하드웨어 기반 신뢰**: TPM 하드웨어의 변조 방지 기능 활용
- **키 계층 구조**: EK → AIK → AK의 계층적 키 관리
- **암호화 알고리즘 검증**: 허용된 알고리즘만 사용 (`accept_tpm_hash_algs` , `accept_tpm_encryption_algs`)

4.3 TPMState 클래스 구조 분석

keylime/agentstates.py 의 TPMState 클래스는 TPM의 PCR 상태를 모델링합니다.

4.3.1 클래스 정의

```
class TPMState:
"""TPMState models the state of the TPM's PCRs"""
pcrs: Dict[int, Optional[bytes]] # PCR 번호별 해시 값 저장
hash_alg: Dict[int, Hash] # PCR별 해시 알고리즘 기록
```

4.3.2 핵심 메서드

4.3.2.1 PCR 초기화

```
def init_pcr(self, pcr_num: int, hash_alg: Hash) -> None:
"""PCR 초기화"""
if pcr_num not in self.hash_alg:
self.hash_alg[pcr_num] = hash_alg
if self.pcrs[pcr_num] is None:
self.pcrs[pcr_num] = TPMState.initial_pcr_value(pcr_num, hash_alg)

@staticmethod
def initial_pcr_value(pcr_num: int, hash_alg: Hash) -> bytes:
"""PCR 초기값 반환"""
```

```
if 17 <= pcr_num <= 23:
return hash_alg.get_ff_hash() # 0xFF로 초기화된 해시
return hash_alg.get_start_hash() # 0x00으로 초기화된 해시
```

4.3.2.2 PCR 상태 관리

```
def get_pcr(self, pcr_num: int) -> Optional[bytes]:
"""PCR 값 조회"""
return self.pcrs.get(pcr_num, None)

def set_pcr(self, pcr_num: int, pcr_value: bytes) -> None:
"""PCR 값 설정"""
self.pcrs[pcr_num] = pcr_value

def used_pcr(self, pcr_num: int) -> bool:
"""PCR 사용 여부 확인"""
return (pcr_num in self.hash_alg) and (self.pcrs[pcr_num] is not None)
```

4.3.3 PCR 번호별 용도

- **PCR 0-7**: BIOS/UEFI 측정값
- **PCR 8-15**: OS 로더 및 OS 측정값
- **PCR 16**: 디버그 정보
- **PCR 17-23**: 동적 측정값 (초기값 0xFF)

4.4 TPMClockInfo 데이터 클래스 역할

4.4.1 클래스 정의

```
@dataclass
class TPMClockInfo:
clock: int # TPM 내부 클록 값
resetcount: int # TPM 리셋 횟수
restartcount: int # TPM 재시작 횟수
safe: int # 안전 상태 플래그 (1=안전, 0=불안전)
```

4.4.2 주요 기능

4.4.2.1 딕셔너리 변환

```
@classmethod
def  from_dict(cls, dclki: Dict[str, int]) -> "TPMClockInfo":
"""딕셔너리에서 TPMClockInfo 객체 생성"""
return  cls(
clock=dclki.get("clock", 0),
resetcount=dclki.get("resetCount", 0),
restartcount=dclki.get("restartCount", 0),
safe=dclki.get("safe", 1),
)
```

```
def to_dict(self) -> Dict[str, int]:
"""TPMClockInfo 객체를 딕셔너리로 변환"""
return {
"clock": self.clock,
"resetCount": self.resetcount,
"restartCount": self.restartcount,
"safe": self.safe,
}
```

4.4.3 보안적 의미

- **clock**: TPM의 단조 증가 카운터로 재생 공격 방지
- **resetcount**: 하드웨어 리셋 감지를 통한 변조 시도 탐지
- **restartcount**: 소프트웨어 재시작 추적
- **safe**: TPM 상태의 안전성 지표

4.5 Agent 등록 및 인증 프로세스

4.5.1 등록 프로세스 단계

4.5.1.1 1단계: Registrar 등록

```
# keylime/registrar_client.py의 getData 함수 사용
def getData(registrar_ip: str, registrar_port: str, agent_id: str,
tls_context: Optional[ssl.SSLContext]) -> Optional[RegistrarData]:
```

```
"""Registrar에서 Agent 데이터 조회"""

# TLS 컨텍스트 필수 검증

if not tls_context:

raise Exception("AIK 쿼리를 위해서는 서버 인증 TLS가 필요합니다.")

# GET /v{version}/agents/{agent_id} 요청

response = client.get(f"/v{API_VERSION}/agents/{agent_id}")
```

4.5.1.2 2단계: RegistrarData 구조

```
class RegistrarData(TypedDict):

ip: Optional[str] # Agent IP 주소

port: Optional[str] # Agent 포트

regcount: int # 등록 횟수

mtls_cert: Optional[str] # mTLS 인증서

aik_tpm: str # AIK 공개키

ek_tpm: str # EK 공개키

ekcert: Optional[str] # EK 인증서

provider_keys: NotRequired[Dict[str, str]] # 제공자 키
```

4.5.2 인증 프로세스

4.5.2.1 mTLS 인증 설정

```
# keylime/tenant.py에서 Agent mTLS 컨텍스트 생성

if  self.registrar_data["mtls_cert"] != "disabled":
```

```
self.verify_custom = self.registrar_data["mtls_cert"]

self.agent_tls_context = web_util.generate_tls_context(

self.client_cert,

self.client_key,

self.trusted_server_ca,

self.client_key_password,

self.verify_server_cert,

is_client=True,

ca_cert_string=self.verify_custom, # Agent 자체 서명 인증서

logger=logger,

)
```

4.5.2.2 API 버전 협상

```
# Agent와 API 버전 협상

with RequestsClient(f"{bracketize_ipv6(self.agent_ip)}:{self.agent_port}",

tls_enabled=self.enable_agent_mtls,

tls_context=tls_context) as get_version:

res = get_version.get("/version")

if res and res.status_code == 200:

data = res.json()

api_version = data["results"]["supported_version"]

if keylime_api_version.validate_version(api_version):
```

```
self.supported_version = api_version
```

4.5.3 보안 검증 단계

4.5.3.1 EK 인증서 검증

1. **EK 인증서 체인 검증**: TPM 제조사 CA를 통한 EK 인증서 유효성 확인
2. **AIK 바인딩**: EK를 사용한 AIK 암호화로 TPM 소유권 증명
3. **MakeCredential/ActivateCredential**: TPM 2.0 표준 자격 증명 프로토콜

4.5.3.2 지속적 검증

```
# AgentAttestState를 통한 상태 추적

class AgentAttestState:

agent_id: str # Agent 식별자

next_ima_ml_entry: int # 다음 IMA 측정 로그 엔트리

boottime: int # 부팅 시간

tmp_clockinfo: TPMClockInfo # TPM 클록 정보

tpm_state: TPMState # TPM 상태

ima_pcrs: Set[int] # IMA PCR 집합

ima_keyring: ImaKeyrings # IMA 키링
```

4.5.4 오류 처리 및 복구

- **연결 재시도**: 지수 백오프를 사용한 연결 재시도 메커니즘
- **인증서 검증 실패**: mTLS 인증서 불일치 시 명확한 오류 메시지 제공
- **TPM 상태 불일치**: PCR 값 불일치 시 재등록 프로세스 시작

4.6 Agent 상태 관리

4.6.1 AgentAttestStates 싱글톤

```
class AgentAttestStates:
"""Agent별 증명 상태를 관리하는 싱글톤 클래스"""
@staticmethod
def  get_instance() -> "AgentAttestStates":
"""싱글톤 인스턴스 반환"""
if not AgentAttestStates.instance:
AgentAttestStates.instance = AgentAttestStates()
return AgentAttestStates.instance
```

4.6.2 상태 동기화

- **스레드 안전성**: `threading.Lock` 을 사용한 동시성 제어
- **상태 지속성**: Agent 재시작 시에도 상태 정보 유지
- **증분 증명**: 이전 상태를 기반으로 한 효율적인 증명 업데이트

이러한 Agent 컴포넌트의 상세 분석을 통해 Keylime 시스템에서 Agent가 어떻게 TPM과 통신하고, 다른 컴포넌트들과 상호작용하며, 시스템 무결성을 보장하는지 이해할 수 있습니다.